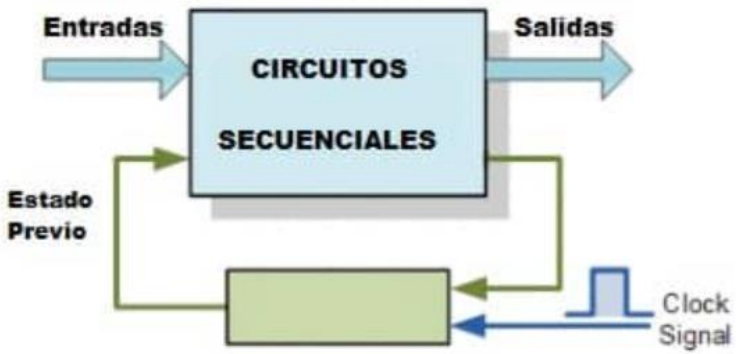
Introducción a VHDL

Matías Javier Oliva
Pablo García

Lógica combinacional vs lógica secuencial

- Las funciones lógicas se pueden dividir en dos:
  - Por un lado están aquellas que solo dependen de los valores actuales de las señales: Lógica combinacional (compuertas and, or, xor, multiplexores…).
  - Por otro lado aquellas que utilizan memoria, es decir dependen de valores actuales y pasados de las señales: Lógica secuencial. Este tipo de lógica se basa en flip-flops (En general el FFD).



Lógica combinacional: Distintas sintaxis

- La lógica combinacional se puede describir de distintas maneras.
- Una opción es describirla con instrucciones concurrentes. Esta es tal vez la forma más directa de describir algo combinacional
- Una descripción de este tipo está dentro de la arquitectura, pero fuera de un proceso.

- Pueden ser cosas simples como:

```
A <= B and C;
```

- O pueden ser instrucciones concurrentes más complejas:

```
A <= '1' when ((B='1') and (C='1')) else '0';
```

Similar al operador ternario en C

Lógica combinacional: Distintas sintaxis

- Generalizado esta sintaxis es una asignación condicional de señales:

```
<optional_label>: <target> <=
<value> when <condition> else
<value> when <condition> else
<value> when <condition> else
...
<value>;
```

- Una sintaxis concurrente alternativa recuerda a una sentencia case:

```
<optional_label>: with <expression> select
<target> <= <value> when <choices>,
          <value> when <choices>,
          <value> when <choices>,
          ...
          <value> when others;
```

Lógica combinacional: Proceso combinacional

- Alternativamente la lógica combinacional se puede describir mediante un proceso.
- Dentro del proceso las instrucciones se pueden pensar como secuenciales, aunque lo que infiere el sintetizador es combinacional.

```
<optional_label>:
process(<sensitivity_list>) is
    -- Declaration(s)
begin
    -- Sequential Statement(s)
end process;
```

- La lista de sensibilidad le dice al sintetizador ante qué cambios de señales tiene que “ejecutar” lo que esta dentro del proceso.

- Dentro del proceso se pueden usar estructuras de control que remiten a lo que conocemos de lenguajes de programación clásicos.

Estructuras de control (dentro de process)

```
if <expression> then
-- Sequential Statement(s)

elsif <expression> then
   -- Sequential Statement(s)

else
   -- Sequential Statement(s);

end if;
```

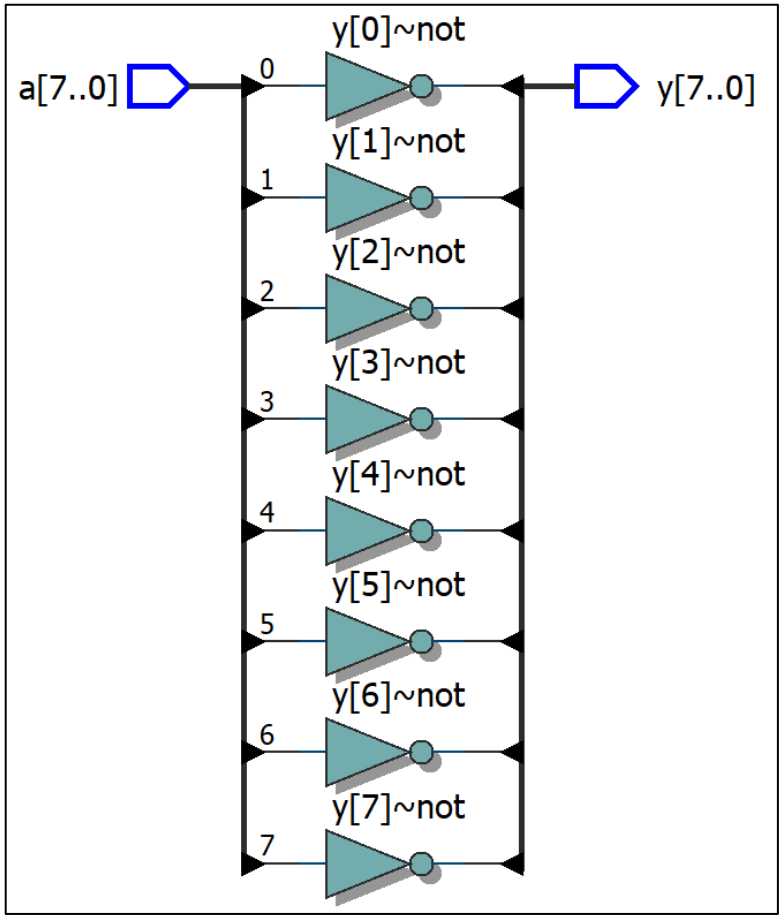
```
case <expression> is
   when <constant_expression> =>
      -- Sequential Statement(s)

   when <constant_expression> =>
      -- Sequential Statement(s)

   when others =>
      -- Sequential Statement(s)

end case;
```

```
<optional_label>:
for <loop_id> in <range> loop
   -- Sequential Statement(s)
end loop;
```

```
<optional_label>:
while <condition> loop
   -- Sequential Statement(s)
end loop;
```

Ejemplo de for loop combinacional

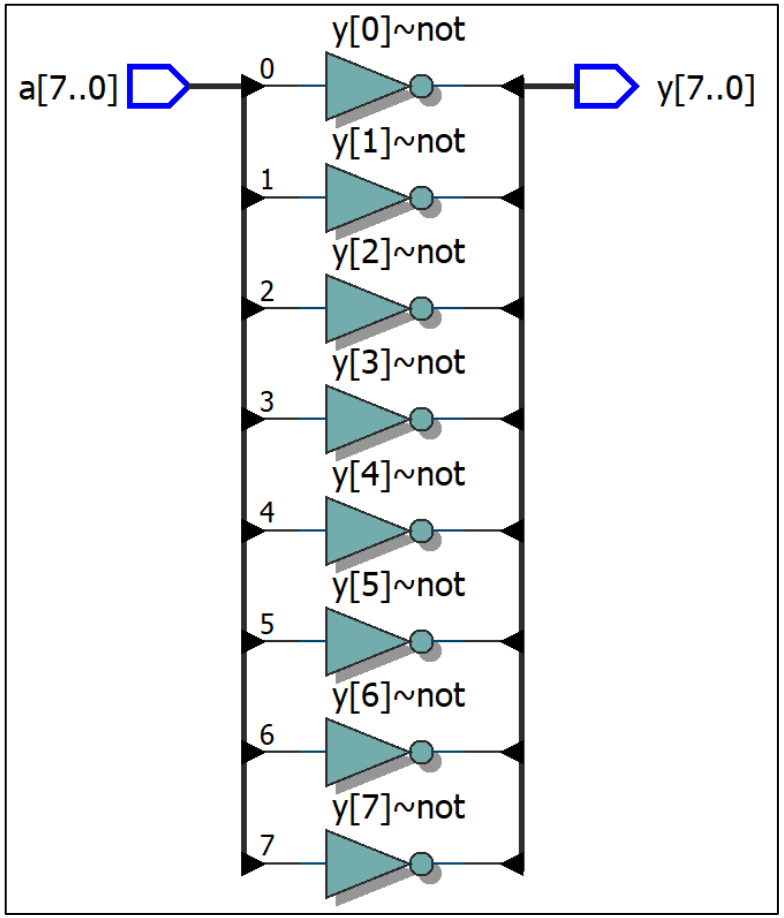
```
library ieee;
use ieee.std_logic_1164.all;

entity Clase2_f is
    port (
        a : in  std_logic_vector(7 downto 0);
        y : out std_logic_vector(7 downto 0)
    );
end entity;

architecture Combinacional of Clase2_f is
begin
    process(a)
    begin
        for i in 0 to 7 loop
            y(i) <= not a(i);
        end loop;
    end process;
end architecture;
```



Ejemplo de while loop combinacional

```
architecture Combinacional of Clase2_f is
begin

    process(a)
        variable i : integer := 0;
    begin
        i := 0;
        while (i < 8) loop
            y(i) <= not(a(i));
            i := i + 1;
        end loop;
    end process;


end architecture;
```

- Lo mismo se puede impelentar con un while-loop.
- Resulta un poco menos natural.



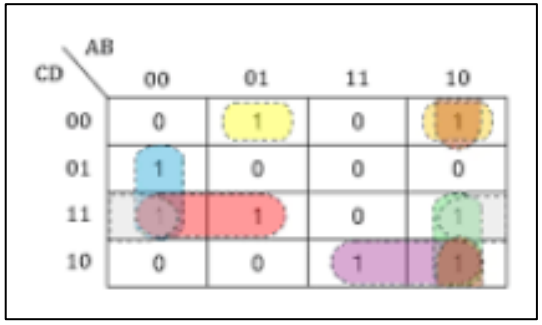
Optimización de expresiones

- Para simplificar expresiones lógicas se utilizan distintas técnicas:
  - Álgebra de boole
  - Tablas de verdad
  - Diagramas de Karnaugh
- Si una expresión no está optimizada el sintetizador hará un esfuerzo por sintetizarla ahorrando celdas lógicas.
- En general conviene optimizar las expresiones antes de darselas al sintetizador para evitar perder control sobre lo que está pasando

a	b	$a \cdot b$
1	1	1
1	0	0
0	1	0
0	0	0

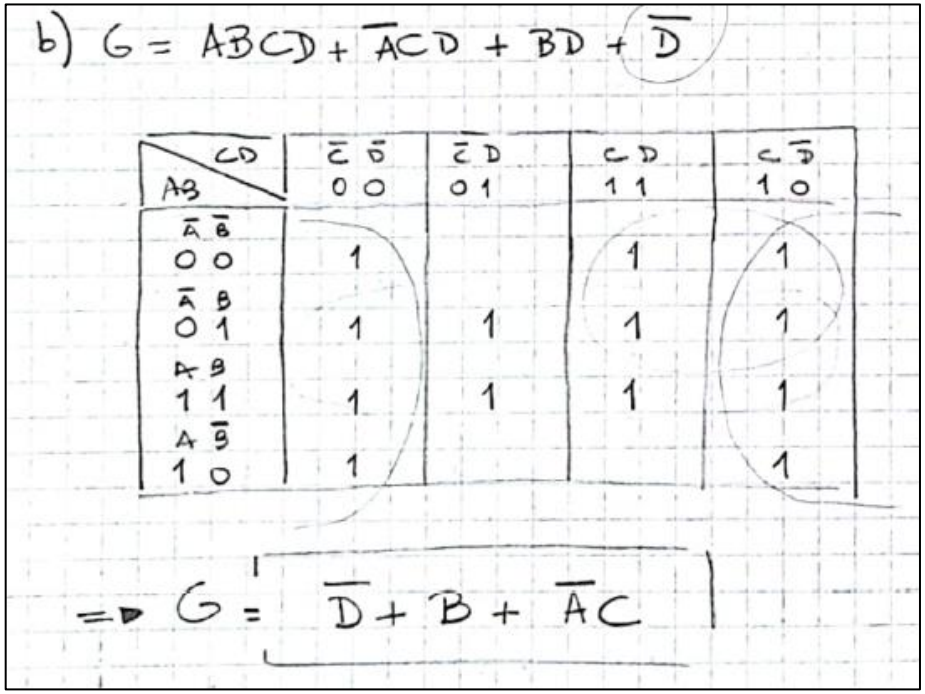
$\longleftrightarrow$

0	0	0
1	1	0
·	1	0



Optimización de expresiones

- Ejemplo con diagrama de Karnaugh:



- No es intención de este curso cubrir los distintos métodos para optimizar expresiones.
- Es bueno tener presente que muchas de estas optimizaciones las realiza el sintetizador, asique a veces la implementación no se condice con lo esperado.

Uso de components: Declaración

- Para modularizar la construcción de sistemas lógicos se utilizan componentes (components).
- Con ellos se puede generar una entidad y después utilizarla como una “caja negra” en otros diseños.

```
component <component_name>

   generic
   (
      <name> : <type>;
      <name> : <type> := <default_value>
   );

   port
   (
      -- Input ports
      <name>   : in  <type>;
      <name>   : in  <type> := <default_value>;

      -- Inout ports
      <name>   : inout <type>;

      -- Output ports
      <name>   : out <type>;
      <name>   : out <type> := <default_value>
   );

end component;
```

Declaración del componente

```
<instance_name> : <component_name>
generic map
(
   <name> => <value>,
   ...
)
port map
(
   <formal_input> => <expression>,
   <formal_output> => <signal>,
   <formal_inout> => <signal>,
   ...
);
```

Instanciación del componente

Uso de components: Ejemplo

- Primero se define la entidad que describe el componente.

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity componente_1 is

   port
   (
      -- Input ports
      A  : in  std_logic;
      B  : in  std_logic;

      -- Output ports
      Y  : out std_logic
   );

end componente_1;


architecture arch of componente_1 is


begin
   Y <= not(B) or ( not(A) and B );
end arch;
```

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity componente_2 is

   port
   (
      -- Input ports
      A  : in  std_logic;
      B  : in  std_logic;

      -- Output ports
      Y  : out std_logic
   );

end componente_2;

architecture arch of componente_2 is

begin
process(A,B)

begin
   if((B = '0') or (A='0' and B='1')) then
      Y <= '1';
   else
      Y <= '0';
   end if;

end process;

end arch;
```

Uso de components: Declaración e Instanciación

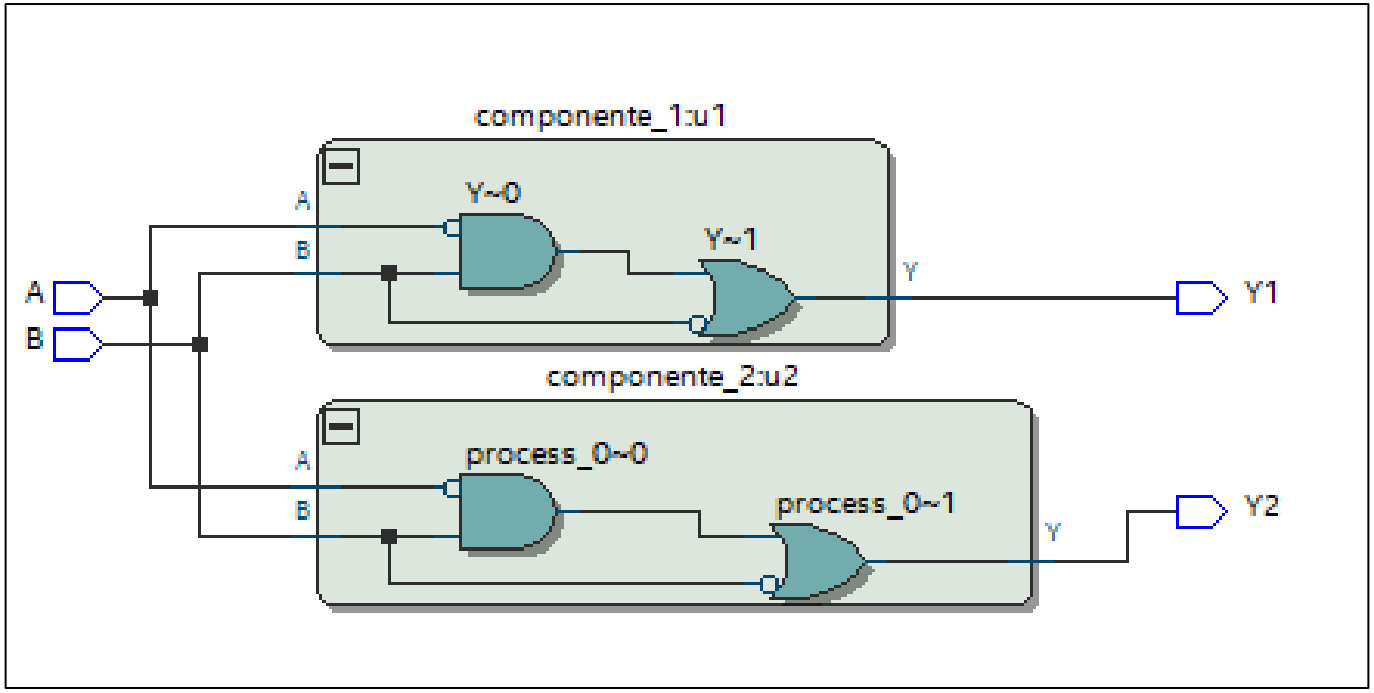
```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity Clase2_a is

   port
   (
      -- Input ports
      A  : in  std_logic;
      B  : in  std_logic;

      -- Output ports
      Y1 : out std_logic;
      Y2 : out std_logic
   );

end Clase2_a;
```

```vhdl
architecture arch of Clase2_a is

component componente_1 is

   port
   (
      -- Input ports
      A  : in  std_logic;
      B  : in  std_logic;

      -- Output ports
      Y  : out std_logic
   );

end component;

component componente_2 is

   port
   (
      -- Input ports
      A  : in  std_logic;
      B  : in  std_logic;

      -- Output ports
      Y  : out std_logic
   );

end component;


begin

   u1: componente_1 port map( A,B,Y1 );
   u2: componente_2 port map( A,B,Y2);


end arch;
```

Declaraciones

Instanciaciones

Uso de components: Diagrama RTL



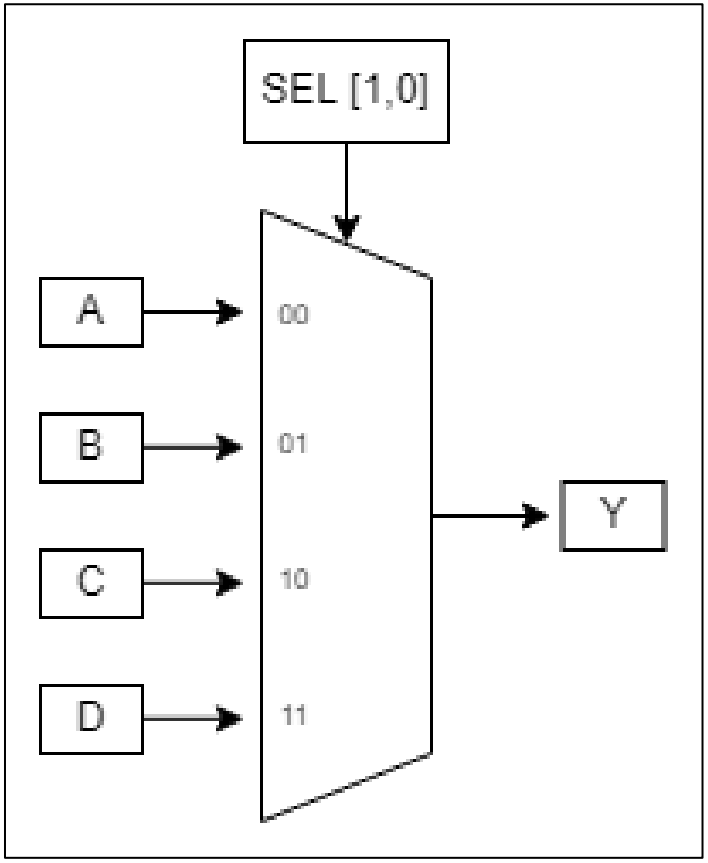
Generación de hardware repetitivo

- La generación de componentes se puede automatizar, para evitar instanciar manualmente muchos componentes.
- Para eso están las estructuras de control for generate e if generate.
- Se usan por fuera de process e instancian una cantidad predeterminada de componentes.
- Son útiles para escribir lógica genérica.
- Por ejemplo podría implementarse una función lógica definida en una entidad N veces.

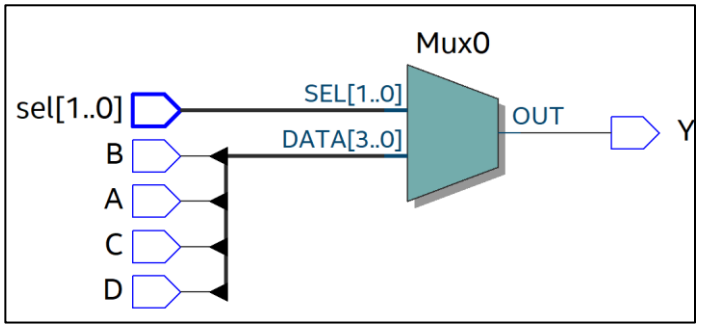
```
<generate_label>:
for <loop_id> in <range> generate
   -- Concurrent Statement(s)
end generate;
```

```
<generate_label>:
if <condition> generate
   -- Concurrent Statement(s)
end generate;
```

Multiplexores



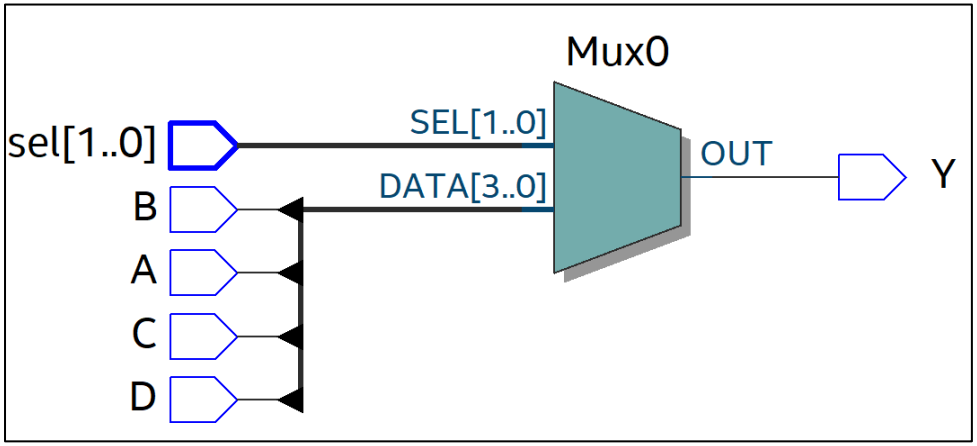

```
architecture arch of Clase2_b is

begin


   with sel select
      Y <= A when "00",
           B when "01",
           C when "10",
           D when "11",
           '0' when others;



end arch;
```



Multiplexores: sintaxis con process

- Lo mismo se puede hacer con process y sentencias case (o de muchas otras formas realmente)

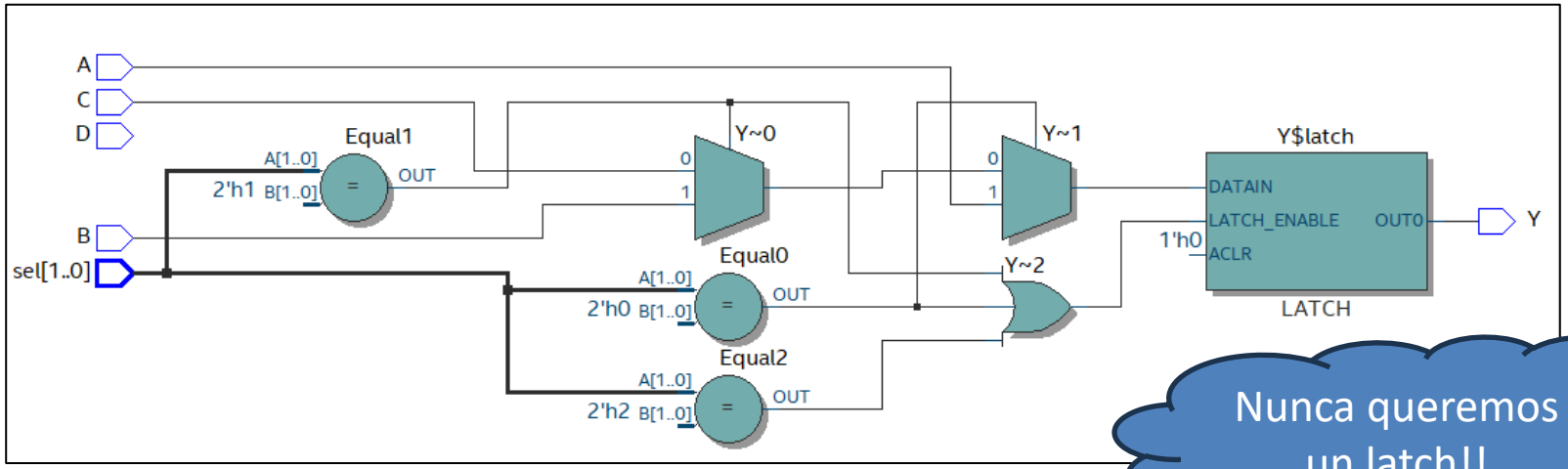
```
process(A,B,C,D)
begin
   case sel is
      when "00" => Y <= A;
      when "01" => Y <= B;
      when "10" => Y <= C;
      when "11" => Y <= D;
      when others => Y <= '0';
   end case;

end process;
```



Inferencia de latches

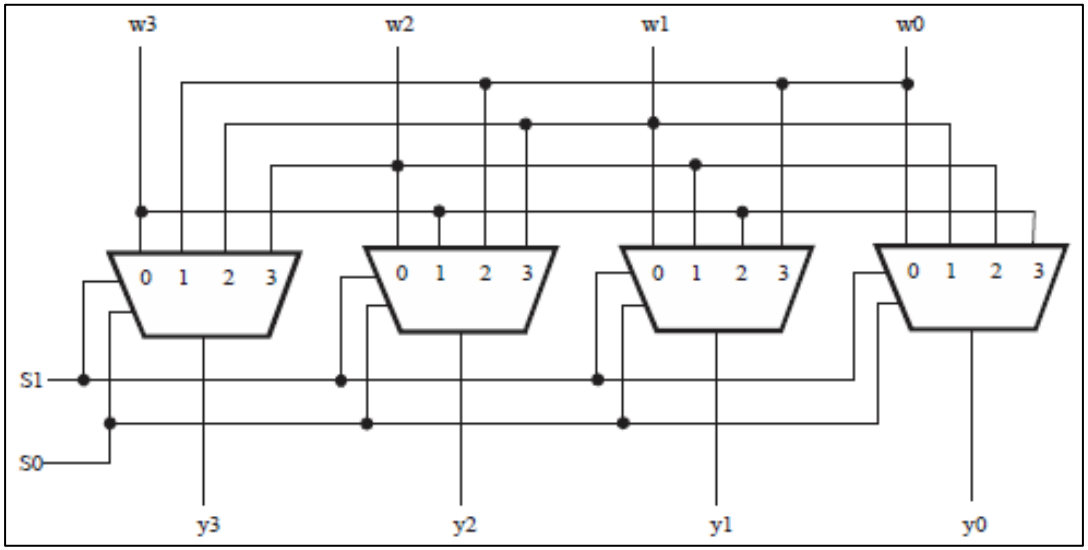
```
process(A,B,C,D, sel)
begin

    if(sel = "00") then
       Y <= A;

    elsif(sel = "01") then
       Y <= B;

    elsif(sel = "10") then
       Y <= C;

    end if;

end process;
```

- Si no se cubren todas las opciones en una estructura condicional como esta se infiere un latch.
- Esto es porque estamos tratando de inferir memoria sin tener un reloj asociado.



Barrel shifter

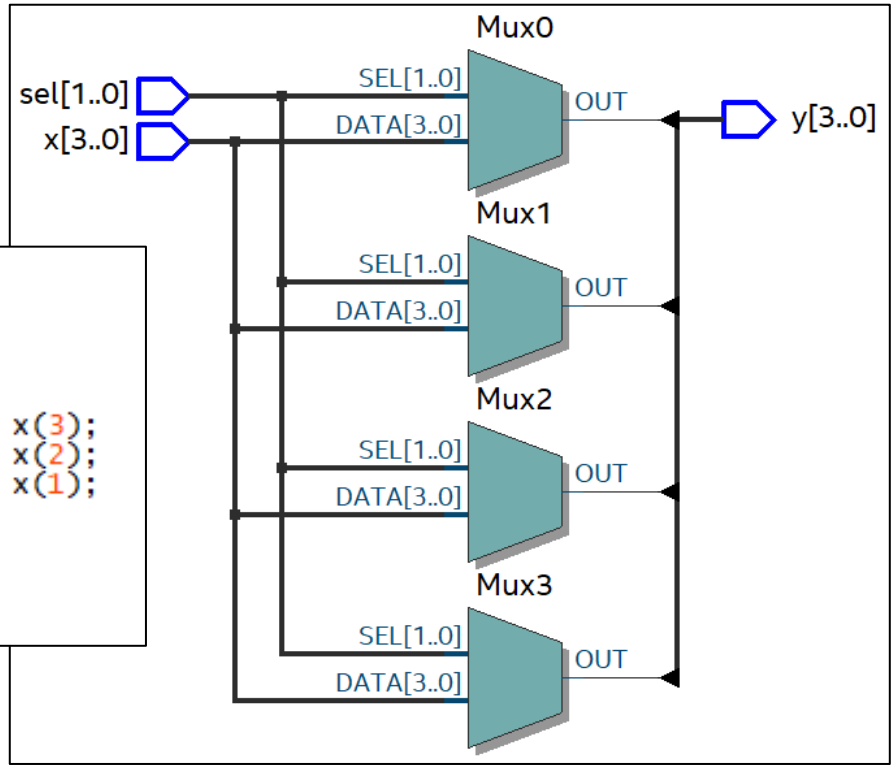
- Permite desplazar una palabra de N bits un número especificado de bits a izquierda o derecha utilizando lógica puramente combinacional (es decir, en un único ciclo de reloj).
- Se puede implementar con multiplexores y lógica adicional



Ejemplo con 4 bits y desplazamiento a derecha [S1,S0] posiciones

Barrel shifter

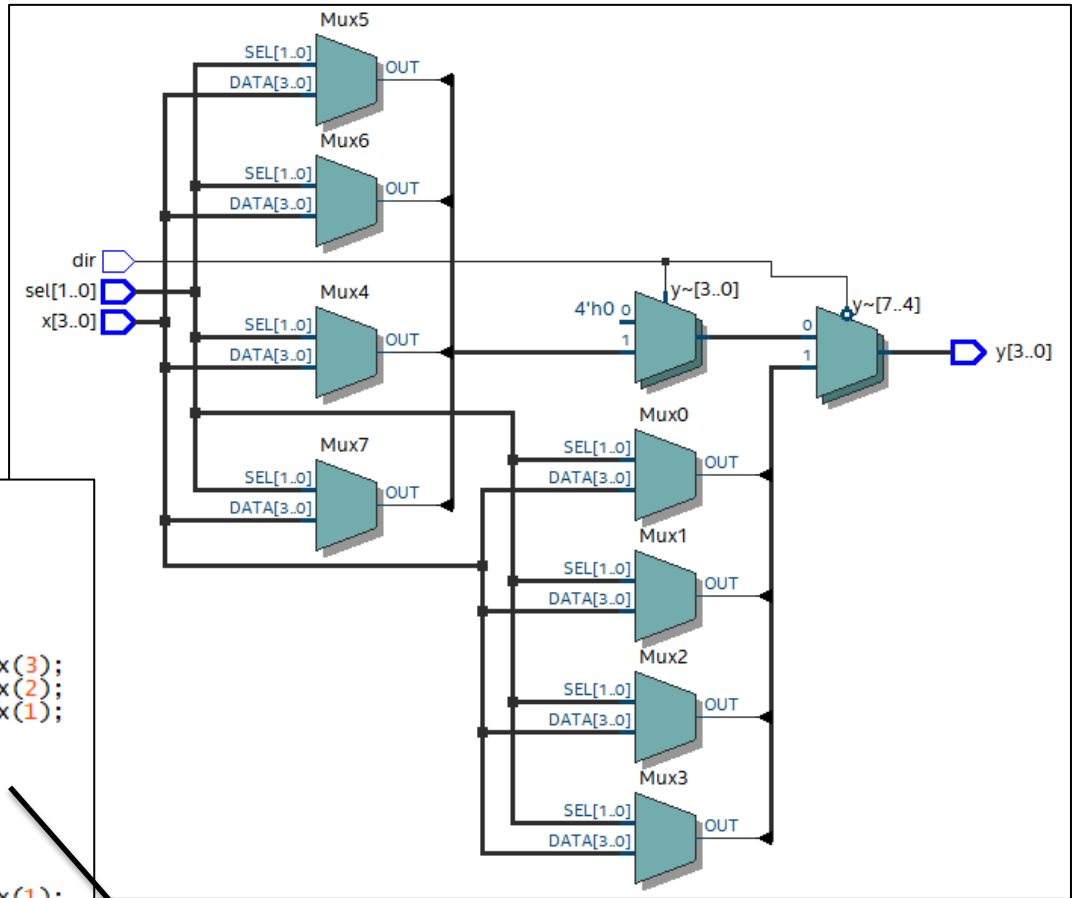
```
process( x,sel )
begin

   case sel is
      when "00" => y <= x;
      when "01" => y <= x(2) & x(1) & x(0) & x(3);
      when "10" => y <= x(1) & x(0) & x(3) & x(2);
      when "11" => y <= x(0) & x(3) & x(2) & x(1);
      when others => y <= "0000";
   end case;

end process;
```



Ejemplo con 4 bits y desplazamiento a izquierda [S1,S0] posiciones

Barrel shifter

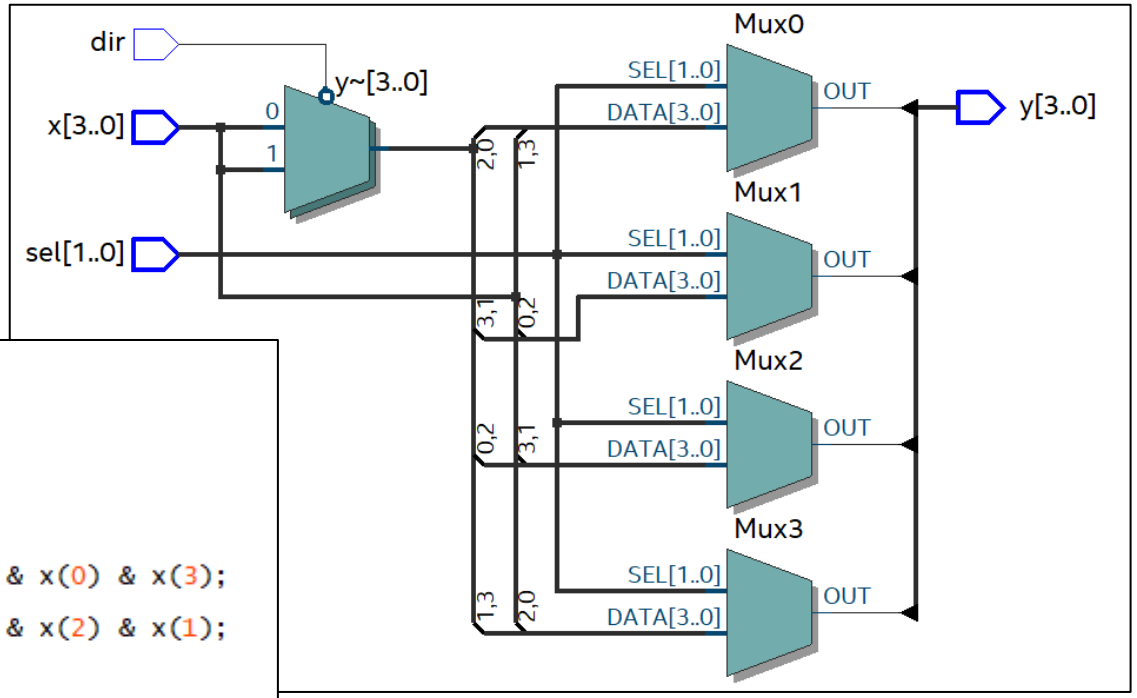
Agregamos desplazamiento a derecha




```
process( x,sel,dir )
begin

   if(dir = '0') then

      case sel is
         when "00" => y <= x;
         when "01" => y <= x(2) & x(1) & x(0) & x(3);
         when "10" => y <= x(1) & x(0) & x(3) & x(2);
         when "11" => y <= x(0) & x(3) & x(2) & x(1);
         when others => y <= "0000";
      end case;

   elsif(dir = '1') then

      case sel is
         when "00" => y <= x;
         when "01" => y <= x(0) & x(3) & x(2) & x(1);
         when "10" => y <= x(1) & x(0) & x(3) & x(2);
         when "11" => y <= x(2) & x(1) & x(0) & x(3);
         when others => y <= "0000";
      end case;

   else y <= "0000";

   end if;

end process;
```

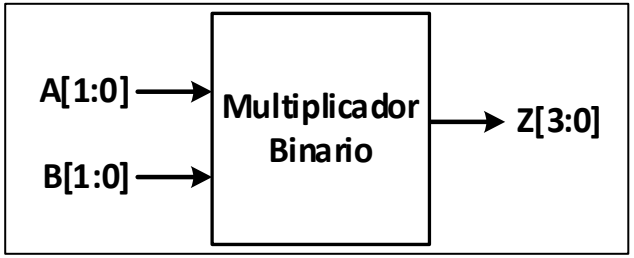
Hay varias condiciones que producen igual salida… puede simplificarse

Barrel shifter




```
process(sel, dir, x)
begin
   case sel is
      when "00" =>
         y <= x;

      when "01" =>
         if dir = '0' then
            y <= x(2) & x(1) & x(0) & x(3);
         else
            y <= x(0) & x(3) & x(2) & x(1);
         end if;

      when "10" =>
         y <= x(1) & x(0) & x(3) & x(2);

      when "11" =>
         if dir = '0' then
            y <= x(0) & x(3) & x(2) & x(1);
         else
            y <= x(2) & x(1) & x(0) & x(3);
         end if;

      when others =>
         y <= "0000";
   end case;
end process;
```

Multiplicador de 2 bits



6	A_1	A_0	B_1	B_0	Z_3	Z_2	Z_1	Z_0	Z
0x0	0	0	0	0	0	0	0	0	0
0x1	0	0	0	1	0	0	0	0	0
0x2	0	0	1	0	0	0	0	0	0
0x3	0	0	1	1	0	0	0	0	0
1x0	0	1	0	0	0	0	0	0	0
1x1	0	1	0	1	0	0	0	1	1
1x2	0	1	1	0	0	0	1	0	2
1x3	0	1	1	1	0	0	1	1	3
2x0	1	0	0	0	0	0	0	0	0
2x1	1	0	0	1	0	0	1	0	2
2x2	1	0	1	0	0	1	0	0	4
2x3	1	0	1	1	0	1	1	0	6
3x0	1	1	0	0	0	0	0	0	0
3x1	1	1	0	1	0	0	1	1	3
3x2	1	1	1	0	0	1	1	0	6
3x3	1	1	1	1	1	0	0	1	9

$$Z_0 = A_0 B_0$$

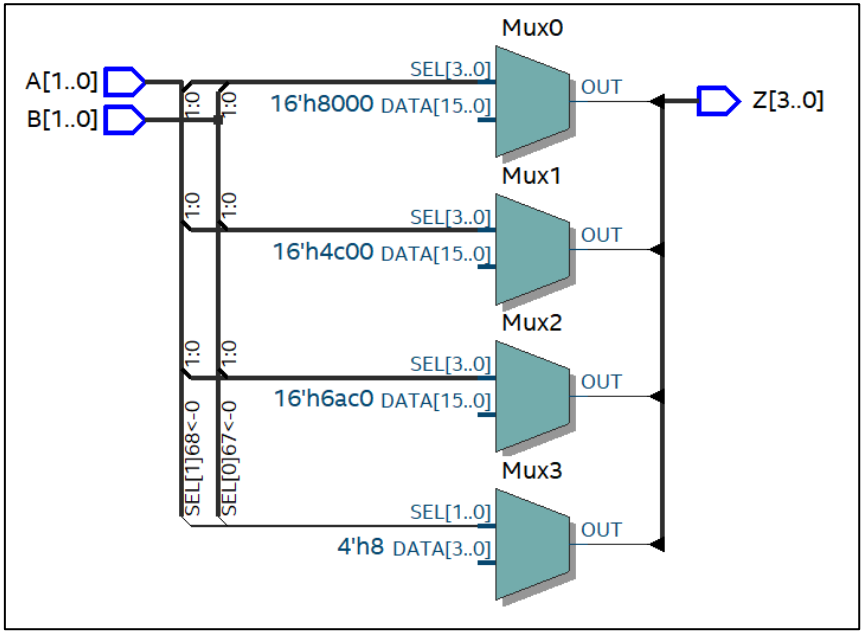
$$Z_1 = A_0 B_1 \overline{B_0} + A_1 \overline{A_0} B_0 + A_0 B_0 (A_1 \oplus B_1)$$

$$Z_2 = A_1 B_1 (\overline{B_0} + \overline{A_0} B_0)$$

$$Z_3 = A_1 A_0 B_1 B_0$$

Multiplicador de 2 bits

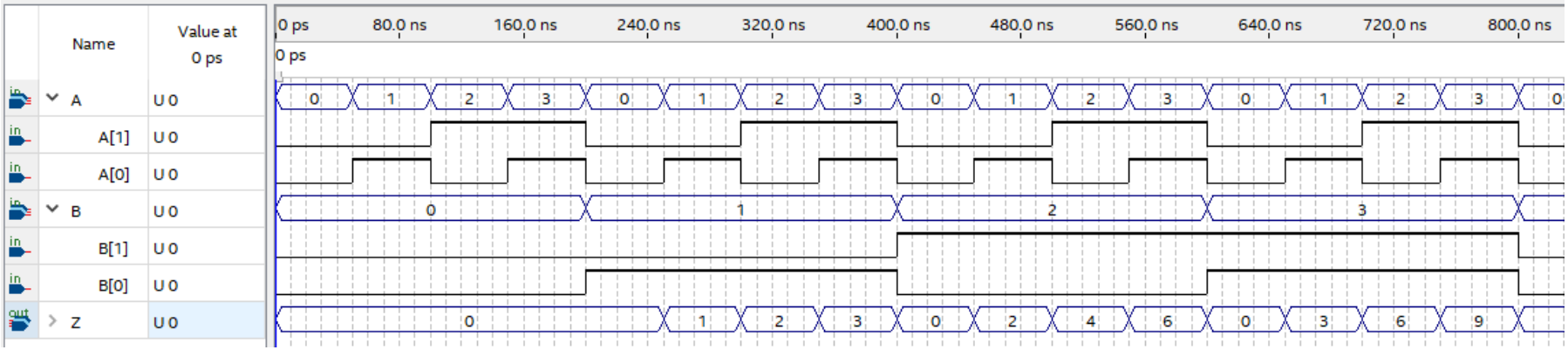
```
architecture alu of Clase2_d is

signal inputs : std_logic_vector (3 downto 0);

signal Z_prod_table_reg : std_logic_vector (3 downto 0);


begin

inputs <= A & B;

p0: process (A,B) is
begin

   case inputs is

      when "0000" =>    Z_prod_table_reg <= "0000";
      when "0001" =>    Z_prod_table_reg <= "0000";
      when "0010" =>    Z_prod_table_reg <= "0000";
      when "0011" =>    Z_prod_table_reg <= "0000";
      when "0100" =>    Z_prod_table_reg <= "0000";
      when "0101" =>    Z_prod_table_reg <= "0001";
      when "0110" =>    Z_prod_table_reg <= "0010";
      when "0111" =>    Z_prod_table_reg <= "0011";
      when "1000" =>    Z_prod_table_reg <= "0000";
      when "1001" =>    Z_prod_table_reg <= "0010";
      when "1010" =>    Z_prod_table_reg <= "0100";
      when "1011" =>    Z_prod_table_reg <= "0110";
      when "1100" =>    Z_prod_table_reg <= "0000";
      when "1101" =>    Z_prod_table_reg <= "0011";
      when "1110" =>    Z_prod_table_reg <= "0110";
      when "1111" =>    Z_prod_table_reg <= "1001";

   end case;

end process;


Z <= Z_prod_table_reg;

end alu;
```



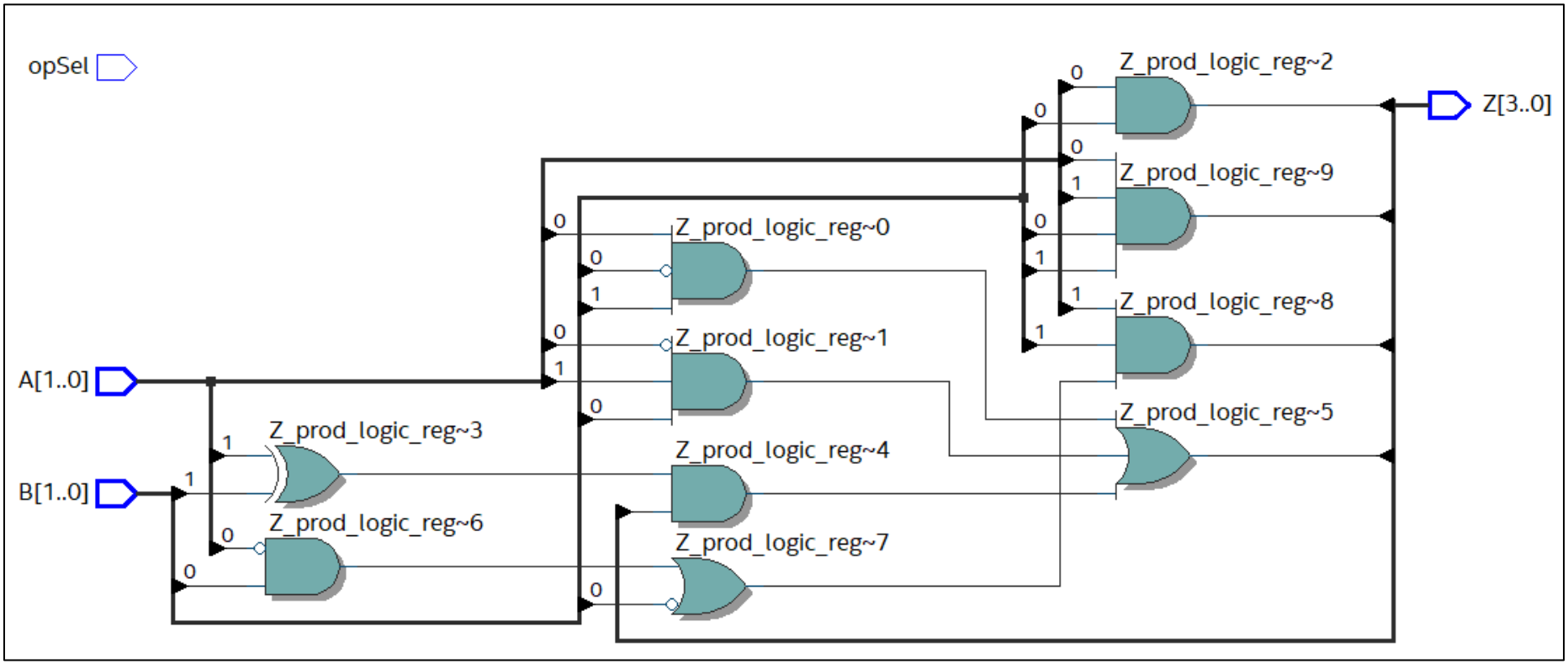
Multiplicador de 2 bits

```
architecture alu of Clase2_d is

signal inputs : std_logic_vector (3 downto 0);

signal Z_prod_logic_reg : std_logic_vector (3 downto 0);

begin

   Z_prod_logic_reg(0) <= A(0) and B(0);

   Z_prod_logic_reg(1) <= (A(0) and B(1) and not(B(0)) ) or
                          (A(1) and not(A(0)) and B(0) ) or
                          (A(0) and B(0) and ( A(1) xor B(1) ) );

   Z_prod_logic_reg(2) <= A(1) and B(1) and
                          (not(B(0)) or ( not(A(0)) and B(0) )) ;

   Z_prod_logic_reg(3) <= A(1) and A(0) and B(1) and B(0);


   Z <= Z_prod_logic_reg;

end alu;
```

Descripción utilizando las funciones lógicas deducidas.



Multiplicador de 2 bits



Comparador de 2 bits

A_1	A_0	B_1	B_0	[C] A>B	[D] A=B	[E] A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

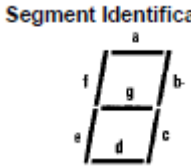
$$\Rightarrow C^{A>B} = A_1\overline{B_1} + A_0\overline{B_1}\,\overline{B_0} + A_1 A_0 \overline{B_0}$$

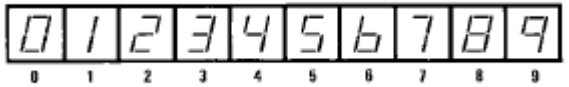
$$\Rightarrow D^{A=B} = \left(\overline{A_0 \oplus B_0}\right)\left(\overline{A_1 \oplus B_1}\right)$$

$$\Rightarrow E^{A<B} = \overline{A_1} B_1 + \overline{A_1}\,\overline{A_0} B_0 + \overline{A_0} B_1 B_0$$

BCD de 7 segmentos

Inputs							Outputs							
LE	$\overline{BI}$	$\overline{LT}$	D	C	B	A	a	b	c	d	e	f	g	Display
X	X	0	X	X	X	X	1	1	1	1	1	1	1	B
X	0	1	X	X	X	X	0	0	0	0	0	0	0	
0	1	1	0	0	0	0	1	1	1	1	1	1	0	0
0	1	1	0	0	0	1	0	1	1	0	0	0	0	1
0	1	1	0	0	1	0	1	1	0	1	1	0	1	2
0	1	1	0	0	1	1	1	1	1	1	0	0	1	3
0	1	1	0	1	0	0	0	1	1	0	0	1	1	4
0	1	1	0	1	0	1	1	0	1	1	0	1	1	5
0	1	1	0	1	1	0	0	0	1	1	1	1	1	6
0	1	1	0	1	1	1	1	1	1	0	0	0	0	7
0	1	1	1	0	0	0	1	1	1	1	1	1	1	8
0	1	1	1	0	0	1	1	1	1	0	0	1	1	9
0	1	1	1	0	1	0	0	0	0	0	0	0	0	
0	1	1	1	0	1	1	0	0	0	0	0	0	0	
0	1	1	1	1	0	0	0	0	0	0	0	0	0	
0	1	1	1	1	0	1	0	0	0	0	0	0	0	
0	1	1	1	1	1	0	0	0	0	0	0	0	0	
0	1	1	1	1	1	1	0	0	0	0	0	0	0	
1	1	1	X	X	X	X				*				*





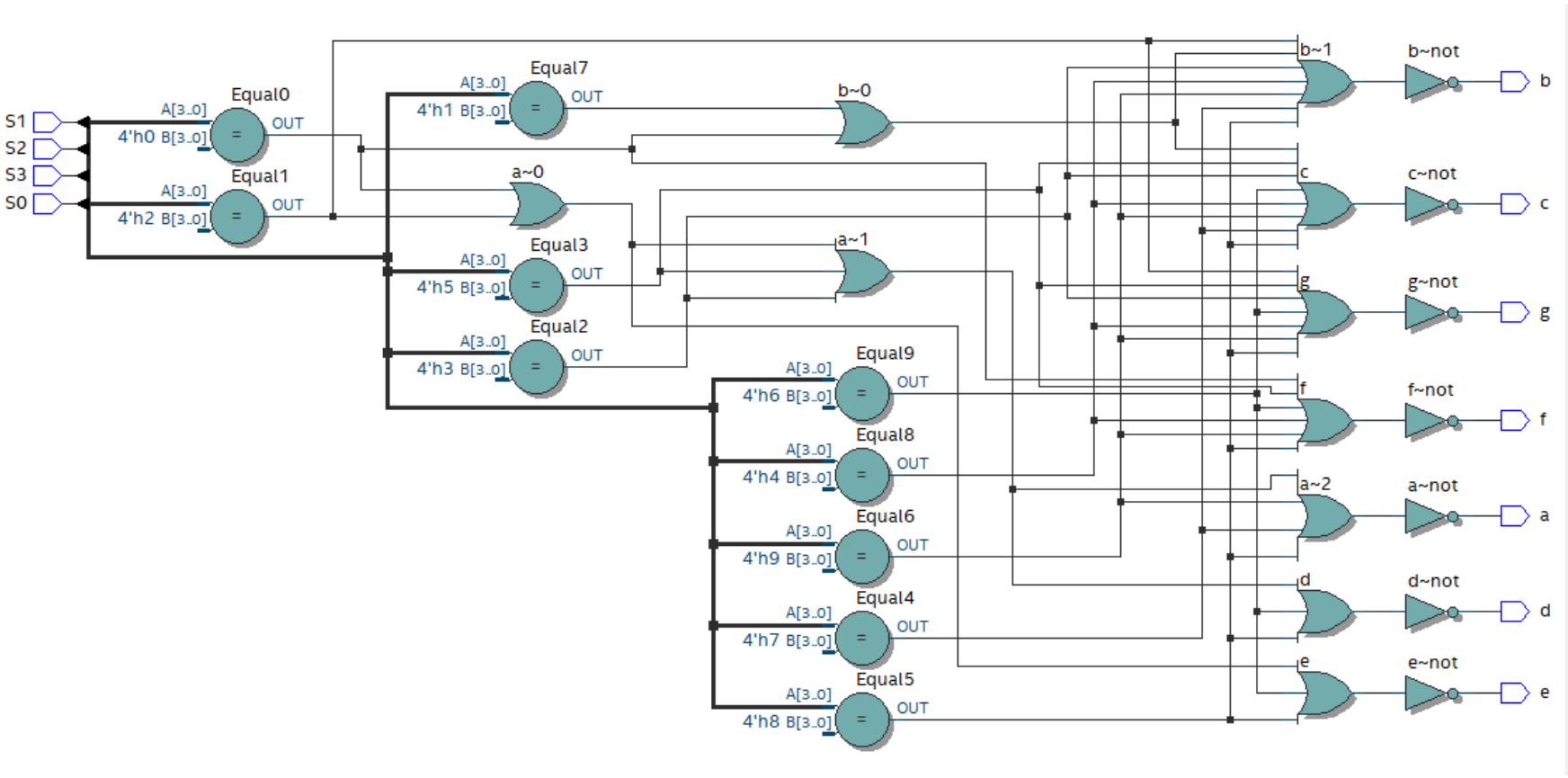
BCD de 7 segmentos

```
library ieee;
use ieee.std_logic_1164.all;

entity Clase2_e is
   port ( S3,S2,S1,S0 : in std_logic;
          a, b, c, d, e, f, g : out std_logic);
end Clase2_e;
```

- Posible implementación con segmentos activos en bajo.
- Implementado mirando directamente la tabla de verdad.
- Existen implementaciones más eficientes, por ejemplo deduciendo las funciones lógicas

```
architecture behaivoral of Clase2_e is
   signal data_in : std_logic_vector (3 downto 0);

begin
   data_in <= S3 & S2 & S1 & S0;


   a <=  '0' when((data_in = "0000") or (data_in = "0010") or
                  (data_in = "0011") or (data_in = "0101") or
                  (data_in = "0111") or (data_in = "1000") or
                  (data_in = "1001") ) else '1';

   b <=  '0' when((data_in = "0000") or (data_in = "0001") or
                  (data_in = "0010") or(data_in = "0011")  or
                  (data_in = "0100") or (data_in = "0111") or
                  (data_in = "1000") or(data_in = "1001")) else '1';

   c <=  '0' when((data_in = "0000") or (data_in = "0001") or
                  (data_in = "0011") or (data_in = "0100") or
                  (data_in = "0101") or (data_in = "0110") or
                  (data_in = "0111") or (data_in = "1000") or
                  (data_in = "1001")) else '1';

   d <=  '0' when((data_in = "0000") or (data_in = "0010") or
                  (data_in = "0011") or (data_in = "0101") or
                  (data_in = "0110") or(data_in = "1000")) else '1';

   e <=  '0' when((data_in = "0000") or (data_in = "0010") or
                  (data_in = "0110") or (data_in = "1000")) else '1';

   f <=  '0' when((data_in = "0000") or (data_in = "0100") or
                  (data_in = "0110") or(data_in = "0101") or
                  (data_in = "1000") or(data_in = "1001")) else '1';

   g <=  '0' when((data_in = "0010") or (data_in = "0011") or
                  (data_in = "0100") or(data_in = "0101") or
                  (data_in = "0110") or(data_in = "1000") or
                  (data_in = "1001")) else '1';

end behaivoral;
```

BCD de 7 segmentos



Uso de generics en los components

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity MuxGenerico is
    generic (
        N : integer := 4  -- Número de entradas
    );
    port (
        inputs : in  std_logic_vector(N-1 downto 0);
        sel    : in  integer range 0 to N-1;
        y      : out std_logic
    );
end entity;

architecture Comportamiento of MuxGenerico is
begin
    process(inputs, sel)
    begin
        y <= '0';
        for i in 0 to N-1 loop
            if sel = i then
                y <= inputs(i);
            end if;
        end loop;
    end process;
end architecture;
```

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity Clase2_g is

   port
   (
      A   : in  std_logic_vector(7 downto 0);
      sel : in  integer range 0 to 7;


      -- Output ports
      B  : out std_logic
);
end Clase2_g;
```

```vhdl
architecture arch of Clase2_g is

component MuxGenerico is
    generic (
        N : integer := 4  -- Número de entradas
    );
    port (
        inputs : in  std_logic_vector(N-1 downto 0);
        sel    : in  integer range 0 to N-1;
        y      : out std_logic
    );
end component;

begin

u0: MuxGenerico generic map (8) port map (A,sel,B);

end arch;
```